

Министерство науки и высшего образования Российской Федерации
Бюджетное образовательное учреждение высшего образования
«Казанский национальный исследовательский технический университет
им. А.Н. Туполева КАИ»

Кафедра систем автоматизированного проектирования

З.М. ГИЗАТУЛЛИН

Методические указания к лабораторным работам
по дисциплине «3D моделирование и техническое зрение»

Казань – 2026

Цель лабораторных работ: получение практических навыков решения задачи технического зрения (определение контуров, обнаружение объектов, идентификации объектов или др.) за счет применения метода (варианты), реализованного в виде законченной пользовательской программы.

1. Теоретическая часть по задаче компьютерного зрения

Компьютерное (техническое) зрение – теория и технология создания машин, которые могут производить обнаружение, отслеживание и классификацию объектов.

Как научная дисциплина, компьютерное зрение относится к теории и технологии создания искусственных систем, которые получают информацию из изображений. Видеоданные могут быть представлены множеством форм, таких как видеопоследовательность, изображения с различных камер или трехмерными данными, например с устройства Kinect или медицинского сканера.

Как технологическая дисциплина, компьютерное зрение стремится применить теории и модели компьютерного зрения к созданию систем компьютерного зрения. Примерами применения таких систем могут быть:

- системы управления процессами (промышленные роботы, автономные транспортные средства);
- системы видеонаблюдения;
- системы организации информации (например, для индексации баз данных изображений);
- системы моделирования объектов или окружающей среды (анализ медицинских изображений, топографическое моделирование); .
- системы взаимодействия (например, устройства ввода для системы человеко-машинного взаимодействия, САПР);
- системы дополненной реальности;

– вычислительная фотография, например, для мобильных устройств с камерами.

Компьютерное зрение также может быть описано как дополнение (но не обязательно противоположность) биологическому зрению. В биологии изучается зрительное восприятие человека и различных животных, в результате чего создаются модели работы таких систем в терминах физиологических процессов. Компьютерное зрение, с другой стороны, изучает и описывает системы компьютерного зрения, которые выполнены аппаратно или программно. Междисциплинарный обмен между биологическим и компьютерным зрением оказался весьма продуктивным для обеих научных областей.

Подразделы компьютерного зрения включают воспроизведение действий, обнаружение событий, слежение, распознавание образов, восстановление изображений и некоторые другие.

Область компьютерного зрения может быть охарактеризована как молодая, разнообразная и динамично развивающаяся. И хотя существуют более ранние работы, можно сказать, что только с конца 1970-х началось интенсивное изучение этой проблемы, когда компьютеры смогли управлять обработкой больших наборов данных, таких как изображения. Однако эти исследования обычно начинались с других областей, и, следовательно, нет стандартной формулировки проблемы компьютерного зрения. Также, и это даже более важно, нет стандартной формулировки того, как должна решаться проблема компьютерного зрения. Вместо этого, существует масса методов для решения различных строго определённых задач компьютерного зрения, где методы часто зависят от задач и редко могут быть обобщены для широкого круга применения. Многие из методов и приложений все ещё находятся в стадии фундаментальных исследований, но всё большее число методов находит применение в коммерческих продуктах, где они часто составляют часть большей системы, которая может решать сложные задачи (например, в области медицинских изображений или измерения и контроля качества в

процессах изготовления). В большинстве практических применений компьютерного зрения компьютеры предварительно запрограммированы для решения отдельных задач, но методы, основанные на знаниях, становятся всё более общими.

Важную часть в области искусственного интеллекта занимает автоматическое планирование или принятие решений в системах, которые могут выполнять механические действия, такие как перемещение робота через некоторую среду. Этот тип обработки обычно нуждается во входных данных, предоставляемых системами компьютерного зрения, действующими как видеосенсор и предоставляющими высокоуровневую информацию о среде и роботе. Другие области, которые иногда описываются как принадлежащие к искусственному интеллекту и которые используются относительно компьютерного зрения, это распознавание образов и обучающие методы. В результате, компьютерное зрение иногда рассматривается как часть области искусственного интеллекта или области компьютерных наук вообще.

Ещё одной областью, связанной с компьютерным зрением, является обработка сигналов. Многие методы обработки одномерных сигналов, обычно временных сигналов, могут быть естественным путём расширены для обработки двумерных или многомерных сигналов в компьютерном зрении. Однако из-за своеобразной природы изображений существует много методов, разработанных в области компьютерного зрения и не имеющих аналогов в области обработки одномерных сигналов. Особым свойством этих методов является их нелинейность, что, вместе с многомерностью сигнала, делает соответствующую подобласть в обработке сигналов частью области компьютерного зрения.

Помимо упомянутых подходов к проблеме компьютерного зрения, многие из исследуемых вопросов могут быть изучены с чисто математической точки зрения. Например, многие методы основываются на статистике, методах оптимизации или геометрии. Наконец, большие работы ведутся в области практического применения компьютерного зрения — того, как существующие

методы могут быть реализованы программно и аппаратно или как они могут быть изменены с тем, чтобы достичь высокой скорости работы без существенного увеличения потребляемых ресурсов.

Компьютерное (техническое) зрение, обработка изображений и машинное зрение – тесно связанные области. Но до сих пор точно не определено, являются ли они разделами одной, более широкой. При детальном анализе может показаться, что это лишь разные названия одной и той же области. Чтобы не возникало путаницы, принято различать их как направления, сфокусированные на определённом предмете изучения. Ниже представлено описание некоторых из них, наиболее важных:

Обработка изображений или анализ изображений, в основном сосредоточены на работе с двухмерными изображениями, то есть как преобразовать одно изображение в другое. Например, попиксельные операции увеличения контрастности, операции по выделению краёв, устранению шумов или геометрические преобразования, такие как Аффинные преобразования. Данные операции предполагают, что обработка/анализ изображения действуют независимо от содержания самих изображений.

Компьютерное зрение сосредотачивается на обработке трехмерных сцен, спроектированных на одно или несколько изображений. Например, восстановлением структуры или другой информации о трехмерной сцене по одному или нескольким изображениям. Компьютерное зрение часто зависит от более или менее сложных допущений относительно того, что представлено на изображениях.

Машинное зрение сосредотачивается на применении, в основном промышленном, например, автономные роботы и системы визуальной проверки и измерений. Это значит, что технологии датчиков изображения и теории управления связаны с обработкой видеоданных для управления роботом и обработка данных в реальном времени осуществляется аппаратно или программно.

Также существует область, названная Визуализация, которая первоначально была связана с процессом создания изображений, но иногда имела дело с обработкой и анализом. Например, рентгенография работает с анализом видеоданных медицинского применения.

Наконец, распознавание образов является областью, которая использует различные методы для получения информации из видеоданных, в основном, основанные на статистическом подходе. Значительная часть этой области посвящена практическому применению этих методов.

Одним из наиболее важных применений является обработка изображений в медицине. Эта область характеризуется получением информации из видеоданных для постановки медицинского диагноза пациентам. В большинстве случаев, видеоданные получают с помощью микроскопии, рентгенографии, ангиографии, ультразвуковых исследований и томографии. Примером информации, которая может быть получена из таких видеоданных является обнаружение опухолей, атеросклероза или других злокачественных изменений. Также примером может быть измерение размеров органов, кровотока и т. д. Эта прикладная область также способствует медицинским исследованиям, предоставляя новую информацию, например, о строении мозга или качеству медицинского лечения.

Другой прикладной областью компьютерного зрения является промышленность. Здесь информацию получают для целей поддержки производственного процесса. Примером может служить контроль качества, когда детали или конечный продукт автоматически проверяются на наличие дефектов. Другим примером является измерение положения и ориентации деталей, поднимаемых рукой робота.

Военное применение является, пожалуй самой большой областью компьютерного зрения. Очевидными примерами являются обнаружение вражеских солдат и транспортных средств и управление ракетами. Наиболее совершенные системы управления ракетами посылают ракету в заданную

область, вместо конкретной цели, а селекция целей производится, когда ракета достигает заданной области, основываясь на получаемых видеоданных. Современное военное понятие, такое как «боевая осведомленность», подразумевает, что различные датчики, включая датчики изображения, предоставляют большой набор информации о поле боя, которая может быть использована для принятия стратегических решений. В этом случае, автоматическая обработка данных используется, чтобы уменьшить сложность или увеличить надежность получаемой информации.

Одними из новых областей применения являются автономные транспортные средства, включая подводные, наземные (роботы, машины), воздушные. Уровень автономности изменяется от полностью автономных (беспилотных) до транспортных средств, где системы, основанные на компьютерном зрении, поддерживают водителя или пилота в различных ситуациях. Полностью автономные транспортные средства используют компьютерное зрение для навигации, то есть для получения информации о месте своего нахождения, для создания карты окружающей обстановки, для обнаружения препятствий. Они также могут быть использованы для определённых задач, например, для обнаружения лесных пожаров. Примерами таких систем могут быть система предупредительной сигнализации о препятствиях на машинах и системы автономной посадки самолетов. Некоторые производители машин демонстрировали системы автономного управления автомобилем, но эта технология все ещё не достигла того уровня, когда её можно запустить в массовое производство.

Другие области применения включают:

Поддержку создания видеоэффектов для кино и телевидения;

Наблюдение;

Типичные задачи компьютерного зрения;

Каждая из областей применения компьютерного зрения, описанных выше, связана с рядом задач; более или менее хорошо определённые проблемы измерения или обработки могут быть решены с использованием множества

методов. Некоторые примеры типичных задач компьютерного зрения представлены ниже.

Распознавание

Классическая задача в компьютерном зрении, обработке изображений и машинном зрении это определение содержат ли видеоданные некоторый характерный объект, особенность или активность. Эта задача может быть достоверно и легко решена человеком, но до сих пор не решена удовлетворительно в компьютерном зрении в общем случае: случайные объекты в случайных ситуациях.

Существующие методы решения этой задачи эффективны только для отдельных объектов, таких как простые геометрические объекты (например, многогранники), человеческие лица, печатные или рукописные символы, автомобили и только в определённых условиях, обычно это определённое освещение, фон и положение объекта относительно камеры.

В литературе описано различное множество проблем распознавания:

Распознавание: один или несколько предварительно заданных или изученных объектов или классов объектов могут быть распознаны, обычно вместе с их двухмерным положением на изображении или трехмерным положением в сцене.

Идентификация: распознается индивидуальный экземпляр объекта. Примеры: идентификация определённого человеческого лица или отпечатка пальцев или автомобиля.

Обнаружение: видеоданные проверяются на наличие определённого условия. Например, обнаружение возможных неправильных клеток или тканей в медицинских изображениях. Обнаружение, основанное на относительно простых и быстрых вычислениях иногда используется для нахождения небольших участков в анализируемом изображении, которые затем анализируются с помощью приемов, более требовательных к ресурсам, для получения правильной интерпретации.

Существует несколько специализированных задач, основанных на распознавании, например:

Поиск изображений по содержанию: нахождение всех изображений в большом наборе изображений, которые имеют определённое содержание. Содержание может быть определено различными путями, например в терминах схожести с конкретным изображением (найдите мне все изображения похожие на данное изображение), или в терминах высокоуровневых критериев поиска, вводимых как текстовые данные (найдите мне все изображения, на которых изображено много домов, которые сделаны зимой и на которых нет машин).

Оценка положения: определение положения или ориентации определённого объекта относительно камеры. Примером применения этой техники может быть содействие руке робота в извлечении объектов с ленты конвейера на линии сборки.

Оптическое распознавание знаков: распознавание символов на изображениях печатного или рукописного текста, обычно для перевода в текстовый формат, наиболее удобный для редактирования или индексации (например, ASCII).

Движение

Несколько задач, связанных с оценкой движения, в которых последовательность изображений (видеоданные) обрабатываются для нахождения оценки скорости каждой точки изображения или 3D сцены. Примерами таких задач являются:

Определение трехмерного движения камеры

Слежение, то есть следование за перемещениями объекта (например, машин или людей)

Восстановление сцены

Даны два или больше изображения сцены, или видеоданные. Восстановление сцены имеет задачей воссоздать трехмерную модель сцены. В простейшем случае, моделью может быть набор точек трехмерного

пространства. Более сложные методы воспроизводят полную трехмерную модель.

Восстановление изображений

Задача восстановления изображений это удаление шума (шум датчика, размытость движущегося объекта и т. д.). Наиболее простым подходом к решению этой задачи являются различные типы фильтров, таких как фильтры нижних или средних частот. Более сложные методы используют представления того, как должны выглядеть те или иные участки изображения, и на основе этого их изменение.

Более высокий уровень удаления шумов достигается в ходе первоначального анализа видеоданных на наличие различных структур, таких как линии или границы, а затем управления процессом фильтрации на основе этих данных.

Системы компьютерного зрения

Реализация систем компьютерного зрения сильно зависит от области их применения, аппаратной платформы и требований по производительности. Некоторые системы являются автономными и решают специфические проблемы детектирования и измерения, тогда как другие системы составляют подсистемы более крупных систем, которые уже могут содержать подсистемы контроля механических манипуляторов (роботы), информационные базы данных (поиск похожих изображений), интерфейсы человек-машина (компьютерные игры) и т. д. Однако, существуют функции, типичные для многих систем компьютерного зрения.

Получение изображений: цифровые изображения получаются от одного или нескольких датчиков изображения, которые помимо различных типов светочувствительных камер включают датчики расстояния, радары, ультразвуковые камеры и т. д. В зависимости от типа датчика, получающиеся данные могут быть обычным 2D изображением, 3D изображением или последовательностью изображений. Значения пикселей обычно соответствуют интенсивности света в одной или нескольких спектральных

полосах (цветные или изображения в оттенках серого), но могут быть связаны с различными физическими измерениями, такими как глубина, поглощение или отражение звуковых или электромагнитных волн, или ядерным магнитным резонансом.

Предварительная обработка: перед тем, как методы компьютерного зрения могут быть применены к видеоданным с тем, чтобы извлечь определённую долю информации, необходимо обработать видеоданные, с тем чтобы они удовлетворяли некоторым условиям, в зависимости от используемого метода. Примерами являются:

Повторная выборка с тем, чтобы убедиться, что координатная система изображения верна

Удаление шума с тем, чтобы удалить искажения, вносимые датчиком

Улучшение контрастности, для того, чтобы нужная информация могла быть обнаружена

Масштабирование для лучшего различения структур на изображении

Выделение деталей: детали изображения различного уровня сложности выделяются из видеоданных. Типичными примерами таких деталей являются:

Линии, границы и кромки

Локализованные точки интереса, такие как углы, капли или точки: более сложные детали могут относиться к структуре, форме или движению.

Детектирование/Сегментация: на определённом этапе обработки принимается решение о том, какие точки или участки изображения являются важными для дальнейшей обработки. Примерами являются:

Выделение определённого набора интересующих точек

Сегментация одного или нескольких участков изображения, которые содержат характерный объект

Высокоуровневая обработка: на этом шаге входные данные обычно представляют собой набор данных, например набор точек или участок изображения, в котором предположительно находится определённый объект. Примерами являются:

Проверка того, что данные удовлетворяют условиям, зависящим от метода и применения.

Оценка характерных параметров, таких как положение или размер объекта.

Классификация обнаруженного объекта по различным категориям.

2. Теоретическая часть по технологии разработки программного обеспечения

Технология разработки программного обеспечения – это совокупность процессов и методов создания программного продукта в заданные сроки и с заданными характеристиками качества.

Программный продукт (ПП) – совокупность компьютерных программ, процедур и связанных с ними документации и данных.

Программный модуль – отдельно компилируемая часть программного кода (программы).

Программный проект (project) – это временное предприятие, предназначенное для создания уникальных продуктов, услуг или результатов.

Методология – система принципов и способов организации процесса разработки программных средств.

Различают два подхода к разработке ПО:

- 1) структурный;
- 2) объектно-ориентированный.

При структурном подходе логическая структура любой программы выражается комбинацией трёх базовых структур: следование, развилка (ветвление), повторение (цикл). Это позволяет создавать ясные, лёгкие для понимания программы.

Объектно-ориентированный подход использует объектную декомпозицию, то есть поведение программы описывается в терминах взаимодействия объектов. Основные концепции ООП (инкапсуляция,

наследование, полиморфизм) позволяют значительно упростить процесс отладки и повторного использования готового программного кода.

С распространением ООП структурный подход не утратил своего значения – методы объектов состоят из базовых конструкций структурного программирования.

Система – это совокупность взаимодействующих компонентов, работающих совместно для достижения определённых целей.

Свойства и поведение системных компонентов влияют друг на друга сложным образом, и корректное функционирование каждого компонента зависит от функционирования многих других компонентов. Системы часто имеют иерархическую структуру, т.е. в качестве компонентов содержат другие системы (подсистемы).

Современный программный продукт представляет собой систему, которая создаётся коллективом разработчиков. Для описания системы, проектирования отдельных компонентов, обсуждения проблем с заказчиком и внутри коллектива применяются различные условные обозначения – нотации.

Нотация – система условных обозначений и правила их применения.

В технологиях разработки программного обеспечения (ТПО) применяются различные виды нотаций: блок-схемы, UML, псевдокод, неформальные обозначения.

Жизненный цикл программных средств

Разделение программы на отдельные модули, использование структурного и объектно-ориентированного подходов упрощают процессы проектирования, программирования и дальнейшего сопровождения программного продукта.

Однако весь процесс создания программного продукта включает ряд дополнительных, неотъемлемых этапов и процессов, объединяемых понятием жизненного цикла программного средства (ЖЦ ПС).

Жизненный цикл – совокупность процессов и этапов развития организмов живой природы, технических систем и продуктов производства от

момента зарождения или появления потребности в их создании до прекращения функционирования или применения.

В обобщённом виде ЖЦ ПС включает следующие этапы:

1. Определение требования к ПС.
2. Анализ требований и создание концепции ПС.
3. Проектирование.
4. Реализация.
5. Сопровождение.
6. Снятие с эксплуатации.

На уровне международных стандартов ЖЦ ПС наиболее полно отражён в стандарте ISO/IEC 12207:1995 «Информационная технология. Процессы жизненного цикла». В России в 2000 г. введён в действие ГОСТ Р ИСО/МЭК 12207–99, содержащий полный аутентичный текст международного стандарта.

Стандарт устанавливает терминологию и общую структуру процессов ЖЦ ПС. Так, согласно стандарту ЖЦ ПС состоит из процессов, каждый процесс разделён на набор работ, а каждая работа представляет собой набор задач.

Таким образом, согласно стандарту, жизненный цикл программного средства – совокупность процессов, работ и задач, включающая в себя разработку, эксплуатацию и сопровождение ПС или системы, охватывающая их жизнь от формулирования концепции до прекращения использования.

Стандарт предназначен, в первую очередь, для регулирования двусторонних отношений в процессе заказа, разработки, поставки и эксплуатации ПП, а также для обеспечения должного уровня качества ПП. На основании стандарта между заказчиком и разработчиком может быть составлен договор, по которому каждая из сторон обязуется выполнить определённый набор работ, состав и содержание которых определён в стандарте.

Кроме того, описанная в стандарте структура ЖЦ ПС может быть использована для планирования процесса разработки конкретного программного продукта. Стандарт не навязывает какую-то определённую последовательность или продолжительность этапов создания ПП и не предлагает конкретных методов разработки. Он лишь фиксирует содержание ЖЦ ПС, на основании которого можно планировать работу по созданию ПС и конкретизировать обязанности участников этой работы с целью получения ПП надлежащего качества.

Процессы, составляющие ЖЦ ПС, делятся в стандарте на следующие группы:

- ☐ основные;
- ☐ вспомогательные;
- ☐ организационные.

Основные процессы это:

- ☐ заказ;
- ☐ поставка;
- ☐ разработка;
- ☐ эксплуатация;
- ☐ сопровождение.

Процесс заказа и поставки в рамках данной дисциплине не рассматриваются.

Процесс разработки состоит из работ и задач, выполняемых разработчиком, то есть организацией, которая проектирует и разрабатывает ПП.

Процесс включает следующие работы:

- 1) подготовка процесса;
- 2) анализ требований к системе;
- 3) проектирование системной архитектуры (с указанием технических и программных средств и ручных операций);
- 4) анализ требований к программным средствам;

5) проектирование программной архитектуры (трансформация требований к программному объекту в описание его общей структуры и составляющих компонентов);

6) техническое проектирование программных средств (уточнение компонентов программного объекта на уровне программных модулей, которые можно программировать, компилировать и тестировать независимо);

7) программирование и тестирование программных средств;

8) сборка программных средств (объединение программных модулей и компонентов в программный объект);

9) квалификационные испытания программных средств;

10) сборка системы (объединение разработанных программных средств с техническими средствами и ручными операциями и, при необходимости, с другими системами);

11) квалификационные испытания системы;

12) ввод в действие программных средств;

13) обеспечение приёмки программных средств.

В соответствии со стандартом в процессе разработки создаётся большое количество документации: требования к системе и отдельным компонентам, эскизные проекты интерфейсов и базы данных, предварительные версии документации пользователя, результаты квалификационных испытаний и др.

Процесс эксплуатации включает оказание помощи и консультации пользователям, определение процедуры получения и документирования сведений о возникающих проблемах.

Процесс сопровождения реализуется при модификациях ПП и соответствующей документации, вызванных возникшими проблемами или потребностями в модернизации.

Вспомогательные процессы ЖЦ ПС являются частью других процессов и предназначены для обеспечения успешной реализации и качества выполнения ПП (проекта). К ним относят:

□ документирование – состоит из набора работ, при помощи которых планируют, разрабатывают, выпускают, редактируют, распространяют документы, в которых нуждаются администраторы, инженеры и пользователи ПП;

□ управление конфигурацией – комплекс методов, направленных на систематический учёт изменений, вносимых разработчиками в ПП на протяжении ЖЦ ПС, сохранение целостности системы после изменений, создание «базовой линии» программных объектов, к которой можно вернуться в случае непредвиденных сложностей (создание резервных копий, контроль исходного кода и др.);

□ обеспечение качества – процесс обеспечения соответствия ПП и процессов ЖЦ утверждённым требованиям;

□ верификация – процесс определения того, что ПП функционирует в полном соответствии с требованиями и условиями, реализованными в предшествующих работах;

□ аттестация – определение полноты соответствия ПС функциональному назначению;

□ совместный анализ – процесс оценки состояний и результатов работ по проекту, когда одна из сторон договора проверяет другую;

□ аудит – определение соответствия требованиям, планам и условиям договора, когда одна из сторон договора проверяет другую;

□ решение проблем.

Стратегии разработки программных средств и систем

Разработка нового ПП всегда связана с неопределённостью: невозможно предвидеть все проблемы, с которыми столкнутся разработчики, для части этих проблем может не существовать готовых решений.

Стратегия – план действий в условиях неопределённости.

На начальном этапе развития вычислительной техники программное обеспечение разрабатывалось по принципу «кодирование – устранение ошибок». Эта стратегия представлена на рисунке 1.

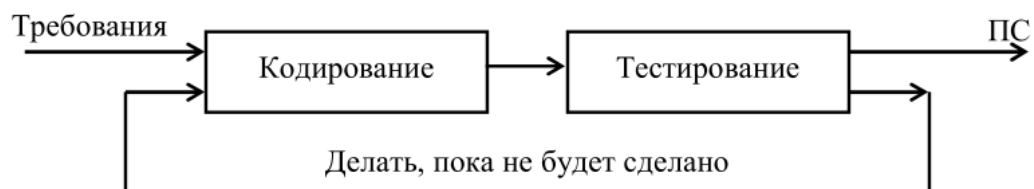


Рисунок 1 – Модель «Делать, пока не будет сделано»

Недостатками данной стратегии являются:

- ☐ неструктурированность процесса разработки ПС;
- ☐ ориентация на индивидуальные знания и умения программиста;
- ☐ сложность управления и планирования;
- ☐ большая длительность и стоимость разработки;
- ☐ низкое качество ПП;
- ☐ высокий уровень рисков проектов.

В настоящее время наиболее широко используются три базовые стратегии разработки ПО:

- ☐ каскадная (водопадная);
- ☐ инкрементная (incremental);
- ☐ эволюционная (iterative).

Выбор той или иной стратегии определяется характеристиками:

- ☐ проекта;
- ☐ требований к продукту;
- ☐ команды разработчиков;
- ☐ возможностями заказчика.

Три базовые стратегии могут быть реализованы различными моделями ЖЦ.

Каскадная стратегия

Каскадная стратегия представляет собой однократный проход этапов разработки.

Она основана на полном определении всех требований к ПС в начале процесса разработки. Возврат к уже выполненным этапам разработки не

предусматривается. Промежуточные результаты в качестве версии программного средства не распространяются.

Основные достоинства каскадной стратегии:

- 1) стабильность требований в течение всего жизненного цикла разработки;
- 2) простота применения стратегии – необходимость только одного прохода этапов разработки;
- 3) простота планирования, контроля и управления проектом;
- 4) возможность достижения высоких требований к качеству ПП в случае отсутствия жестких ограничений затрат и графика работ;
- 5) доступность для понимания заказчиками.

К недостаткам каскадной стратегии следует отнести:

- 1) сложность четкого формулирования требований к ПП в начале ЖЦ и невозможность их динамического изменения на протяжении ЖЦ;
- 2) линейная структура процесса разработки – на практике разрабатываемые ПС обычно слишком велики, чтобы все работы по их созданию выполнить однократно; в результате возврат к предыдущим шагам для решения возникающих проблем приводит к увеличению затрат и нарушению графика работ;
- 3) непригодность промежуточного продукта для использования;
- 4) недостаточное участие пользователя в разработке ПП – только в самом начале (при разработке требований) и в конце (во время приемочных испытаний), что приводит к невозможности предварительной оценки пользователем качества ПС.

Области применения каскадной стратегии:

- 1) при разработке проектов с четкими, неизменяемыми в течение ЖЦ требованиями;
- 2) при разработке ПП такого же типа, который уже разрабатывался ранее;

3) при разработке проекта, связанного с созданием и выпуском новой версии уже существующего продукта или системы;

4) при разработке проекта, связанного с переносом уже существующего продукта на новую платформу.

Инкрементная стратегия

Существуют ситуации, когда требования к ПП достаточно полно определены, однако его последовательная разработка в одном длительном цикле каскадной стратегии порождает дополнительные сложности. Или же необходимо быстро предоставить заказчику ПС с некоторым набором базовых, основных функций и далее расширять функционал в процессе разработки.

Инкрементная стратегия представляет собой многократный проход этапов разработки с запланированным улучшением результата.

Данная стратегия основана на полном определении всех требований к ПС в начале процесса разработки. Однако полный набор требований реализуется постепенно в соответствии с планом в последовательных циклах разработки.

Результат каждого цикла называется инкрементом.

Первый инкремент реализует базовые функции ПС. В последующих инкрементах функции ПС постепенно расширяются, пока не будет реализован весь набор требований к ПС.

Результат каждого цикла разработки может распространяться в качестве очередной поставляемой версии ПП.

Инкрементная стратегия обычно основана на объединении элементов каскадной модели и прототипирования, которое позволяет существенно сократить продолжительность разработки каждого инкремента и всего проекта в целом.

Под прототипом понимается легко поддающаяся модификации и расширению рабочая модель ПС, позволяющая пользователю получить представление о ключевых свойствах ПС до его полной реализации.

Основными достоинствами инкрементной стратегии являются:

- 1) возможность получения функционального продукта после реализации каждого инкремента;
- 2) короткая продолжительность создания инкремента (за счет небольших функциональных различий между соседними инкрементами и за счет использования разработанных ранее компонентов);
- 3) предотвращение реализации громоздких спецификаций требований; стабильность требований во время создания определенного инкремента (за счет короткой продолжительности цикла разработки и возможности переноса неважных изменений на последующие инкременты); возможность учета изменившихся требований;
- 4) снижение рисков по сравнению с каскадной моделью;
- 5) включение в процесс пользователей, что позволяет оценить самые важные функциональные возможности продукта на более ранних этапах разработки и в конечном итоге приводит к повышению качества ПП, снижению затрат и времени на его разработку.

К недостаткам инкрементной стратегии следует отнести:

- 1) необходимость полного функционального определения системы или программного средства в начале ЖЦ для обеспечения определения инкрементов, планирования и управления проектом;
- 2) возможность текущего изменения требований к системе или программному средству, которые уже реализованы в предыдущих инкрементах;
- 3) сложность планирования и распределения работ;
- 4) наличие тенденции к оттягиванию решения трудных проблем на поздние инкременты, что может нарушить график работ.

Области применения инкрементной стратегии:

- 1) при разработке проектов, в которых большинство требований можно сформулировать заранее, но часть из них могут быть определены через определенный период времени;

- 2) при необходимости быстро поставить на рынок продукт, имеющий функциональные базовые свойства;
- 3) при выполнении проекта с применением новой технологии;
- 4) при разработке проектов, для которых разработка системы за один цикл связана с большой степенью риска.

Эволюционная стратегия

Эволюционная стратегия представляет собой многократный проход этапов разработки. Данная стратегия основана на частичном определении требований к ПС в начале процесса разработки. Требования постепенно уточняются в последовательных циклах разработки.

Результат каждого цикла разработки обычно представляет собой очередную поставляемую версию ПС.

В общем случае для эволюционной стратегии характерно существенно меньшее количество циклов разработки при большей продолжительности цикла по сравнению с инкрементной стратегией. При этом результат каждого цикла разработки (очередная версия ПС) гораздо сильнее отличается от результата предыдущего цикла.

Для обеспечения полного понимания требований к ПС применяют их итеративное уточнение с помощью прототипов. В этом случае активное участие в цикле разработки должен принимать заказчик.

Основными достоинствами эволюционной стратегии являются:

- 1) возможность уточнения и внесения новых требований в процессе разработки;
- 2) пригодность для использования промежуточного продукта;
- 3) возможность управления рисками;
- 4) обеспечение широкого участия пользователя в проекте, начиная с ранних этапов, что минимизирует возможность разногласий между заказчиками и разработчиками и обеспечивает создание продукта высокого качества;
- 5) реализация преимуществ инкрементной и каскадной стратегий.

К недостаткам эволюционной стратегии следует отнести:

1) неизвестность точного количества итераций и сложность определения критериев для продолжения процесса разработки на следующей итерации – это

затрудняет планирование проекта и может вызвать задержку реализации конечной версии ПП;

2) необходимость активного участия пользователей в проекте, что не всегда осуществимо;

3) необходимость в обработке дополнительной документации за счет большого количества промежуточных циклов.

Области применения эволюционной:

1) при разработке проектов, для которых требования слишком сложны, неизвестны заранее, непостоянны или требуют уточнения;

2) при разработке проектов, для которых нужна проверка концепции, демонстрация технической осуществимости или промежуточных продуктов;

3) при разработке проектов по созданию новых, не имеющих аналогов продуктов или систем;

4) при отсутствии у разработчиков уверенности в выборе оптимальной архитектуры или применяемых алгоритмов;

5) при разработке проектов, использующих новые технологии.

Классические методологии разработки программных средств

Структурное программирование

Достоинства структурного программирования по сравнению с интуитивным неструктурным программированием :

1) уменьшение трудностей тестирования программ;

2) повышение производительности труда программистов;

3) повышение ясности и читабельности программ, что упрощает их сопровождение;

4) повышение эффективности объектного кода программ как с точки зрения времени их выполнения, так и с точки зрения необходимых затрат памяти.

Основные положения структурного программирования

К концепциям структурного программирования относятся:

- ☐ отказ от использования оператора безусловного перехода (GoTo);
- ☐ применение фиксированного набора управляющих конструкций;
- ☐ использование метода нисходящего проектирования.

В основу структурного программирования положено требование, чтобы каждый модуль алгоритма (программы) проектировался с единственным входом и единственным выходом. Программа представляется в виде множества вложенных модулей, каждый из которых имеет один вход и один выход.

Любая программа может быть выражена комбинацией трех базовых структур:

- 1) следование;
- 2) развилка;
- 3) повторение.

Функциональный блок – это отдельный вычислительный оператор или любая последовательность вычислений с единственным входом и единственным выходом.

Всякая последовательность функциональных блоков называется конструкцией следования.

Развилка (конструкция принятия двоичного (дихотомического) решения) называется также конструкцией If-Then-Else (Если-То-Иначе). Эта структура, обеспечивает выбор между двумя альтернативными путями вычислительного процесса в зависимости от результата проверки некоторого условия.

Повторение (конструкция обобщенного цикла) – в качестве базовой конструкции структурного программирования используется цикл с

предусловием, называемый циклом «Пока» (пока условие истинно, тело цикла выполняется).

Преобразования логических блоков и конструкции следования в один функциональный блок называются преобразованиями Бома–Джакопини. Их основу составляет принцип «чёрного ящика» (часть алгоритма или программы, реализующая некоторую функцию, с одним входом и одним выходом).

Таким образом, всякая программа, состоящая из функциональных блоков, операторов цикла с предусловием и операторов If-Then-Else, поддаётся последовательному преобразованию к единственному функциональному блоку. Эта последовательность преобразований может быть использована как средство понимания программы, подход к доказательству ее правильности и структурированности. Обратная последовательность преобразований может быть использована в процессе проектирования алгоритма (программы) по методу нисходящего проектирования – алгоритм (программа) разрабатывается, исходя из единственного функционального блока, который постепенно детализируется в сложную структуру основных элементов.

Реализация основ структурного программирования при разработке программ на конкретных языках программирования базируется на следующих правилах: все операторы в программе должны представлять собой либо непосредственно исполняемые в линейном порядке функциональные операторы, либо следующие управляющие конструкции :

- 1) вызовы подпрограмм – любое допустимое на конкретном языке программирования обращение к замкнутой подпрограмме с одним входом и одним выходом;

- 2) вложенные на произвольную глубину операторы If-Then-Else;

- 3) циклические операторы (цикл с предусловием).

Этих средств достаточно для составления структурированных программ. Однако иногда допускаются расширения данных конструкций:

- 1) дополнительные конструкции организации цикла:
 - цикл с параметром (цикл «For») как вариант цикла с предусловием;
 - цикл с постусловием, называемый в структурном программировании циклом «До», в котором тело цикла выполняется перед проверкой условия выхода из цикла и повторяется до выполнения условия;
- 2) использование оператора Case как расширения конструкции If-Then-Else;
- 3) подпрограммы с несколькими входами или несколькими выходами (например один выход нормальный, второй – по ошибке), если это допускается отраслевыми стандартами или стандартами предприятия;
- 4) применение оператора GoTo с жёсткими ограничениями (например, передача управления не далее чем на десять операторов или только вперёд по программе в соответствии с положениями отраслевых стандартов или стандартов предприятия).

Модульное проектирование

При разработке модульных ПС могут использоваться методы структурного проектирования или методы объектно-ориентированного проектирования. Их целью является формирование структуры создаваемой программы – ее разделение по некоторым установленным правилам на структурные компоненты (модуляризация) с последующей иерархической организацией данных компонентов. Для различных языков программирования такими компонентами могут быть подпрограммы, внешние модули, объекты и т.п.

Модульная программа – это программа, в которой любую часть логической структуры можно изменить, не вызывая изменений в ее других частях.

Признаки модульности программ:

- 1) программа состоит из модулей.
- 2) модули являются независимыми. Это значит, что модуль можно изменять или модифицировать без последствий в других модулях;

3) условие «один вход – один выход». Модульная программа состоит из модулей, имеющих одну точку входа и одну точку выхода. В общем случае может быть более одного входа, но важно, чтобы точки входов были определены и другие модули не могли входить в данный модуль в произвольной точке.

Достоинства модульного проектирования:

- 1) упрощение разработки ПС;
- 2) исключение чрезмерной детализации обработки данных;
- 3) упрощение сопровождения ПС;
- 4) облегчение чтения и понимания программ;
- 5) облегчение работы с данными, имеющими сложную структуру.

Недостатки модульности:

- 1) модульный подход требует большего времени работы центрального процессора (в среднем на 5 – 10 %) за счет времени обращения к модулям;
- 2) модульность программы приводит к увеличению ее объема (в среднем на 5 – 10 %);
- 3) модульность требует дополнительной работы программиста и определенных навыков проектирования ПС.

Классические методы структурного проектирования модульных ПС делятся на три основные группы :

- 1) методы нисходящего проектирования;
- 2) методы расширения ядра;
- 3) методы восходящего проектирования.

На практике обычно применяются различные сочетания этих методов.

Основное назначение нисходящего проектирования – служить средством разбиения большой задачи на меньшие подзадачи так, чтобы каждую подзадачу можно было рассматривать независимо.

Суть метода нисходящего проектирования заключается в следующем.

На начальном шаге в соответствии с общими требованиями к программному средству разрабатывается его укрупненная структура без

детальной проработки отдельных частей. Затем выделяются функциональные требования более низкого уровня и в соответствии с ними разрабатываются отдельные компоненты программного средства. Каждый из компонентов детализируется до тех пор, пока его составные части не будут окончательно уточнены.

На каждом шаге нисходящего проектирования делается оценка правильности вносимых уточнений в контексте правильности функционирования разрабатываемого программного средства в целом.

Компоненты нижнего уровня ПС называются *программными модулями*.

Классической *стратегией*, на которой основана реализация метода нисходящего проектирования, является *пошаговое уточнение*.

Пошаговое уточнение

При пошаговом уточнении на каждом следующем этапе декомпозиции детализируются программные компоненты очередного более низкого уровня. При этом результаты каждого этапа являются уточнением результатов предыдущего этапа лишь с небольшими изменениями.

Существуют различные способы реализации пошагового уточнения. Далее рассматриваются два классических способа:

- 1) проектирование программного средства с помощью псевдокода и управляющих конструкций структурного программирования;
- 2) использование комментариев для описания обработки данных.

Пошаговое уточнение требует, чтобы взаимное расположение строк текста программы обеспечивало ее читабельность. *Общее правило записи текста разрабатываемой программы*: служебные слова, которыми начинается и заканчивается та или иная управляющая конструкция, записываются на одной вертикали; все вложенные в данную конструкцию псевдокоды (или комментарии, или операторы программы) и управляющие конструкции записываются с отступом вправо.

Преимущества метода пошагового уточнения:

1) основное внимание при его использовании обращается на проектирование корректной структуры программы, а не на ее детализацию;

2) так как каждый последующий этап является уточнением предыдущего лишь с небольшими изменениями, то легко может быть выполнена проверка корректности процесса разработки на всех этапах.

Недостаток метода пошагового уточнения: на поздних этапах проектирования программного средства может обнаружиться необходимость в структурных изменениях, требующих пересмотра более ранних решений.

Использование комментариев для описания обработки данных

При этом способе на каждом этапе уточнений используются *управляющие конструкции структурного программирования*, а *правила обработки данных не детализируются*. Они описываются в виде комментариев.

На каждом этапе уточнений блоки, представленные комментариями, частично детализируются. Но сами комментарии при этом не выбрасываются. В результате после окончания проектирования получается хорошо прокомментированный текст программы.

Наиболее часто используются следующие ***виды комментариев***:

1) *заголовки* – объясняют назначение основных компонентов программы на отдельных этапах пошаговой детализации;

2) *построчные* – описывают мелкие фрагменты программы;

3) *вводные* – помещаются в начале текста программы и задают общую информацию о ней (например, назначение программы, сведения об авторах, дата написания, используемый метод решения, время выполнения, требуемый объем памяти).

Методы восходящего проектирования

При использовании восходящего проектирования в первую очередь выделяются функции нижнего уровня, которые должно выполнять программное средство. Эти функции реализуются с помощью программных модулей самых нижних уровней. Затем на основе этих модулей

проектируются программные компоненты более высокого уровня. Процесс продолжается, пока не будет завершена разработка всего программного средства.

В чистом виде метод восходящего проектирования используется крайне редко. Основным его *недостатком* является то, что программисты начинают разработку программного средства с несущественных, вспомогательных деталей. Это затрудняет проектирование программного средства в целом.

Метод восходящего проектирования *целесообразно* применять в следующих случаях:

- существуют разработанные модули, которые могут быть использованы для выполнения некоторых функций разрабатываемой программы;

- заранее известно, что некоторые простые или стандартные модули потребуются нескольким различным частям программы (например, подпрограмма анализа ошибок, ввода-вывода и т.п.).

Обычно используется *сочетание* методов нисходящего и восходящего проектирования. Такое сочетание возможно различными *способами*. Ниже рассмотрены два из них.

Первый способ сочетания. Выделяются ключевые (наиболее важные) модули промежуточных уровней разрабатываемой программы. Затем проектирование ведется нисходящим и восходящим методами одновременно.

Второй способ сочетания . Проектируются модули нижнего уровня (те, которые необходимо спроектировать заранее). Затем программа проектируется одновременно нисходящим и восходящим методами. При таком способе проектирования наиболее важной задачей является согласование интерфейса между верхними и нижними уровнями программы, выполняемое в последнюю очередь. Это является существенным недостатком данного способа сочетания. Разработчики должны обладать достаточно высокой квалификацией, чтобы не оказалось, что верхняя и нижняя части программы несовместимы между собой.

3. Этапы выполнения лабораторных работ:

1. Лабораторная работа 1. (4 часа)

Выбрать самостоятельно (или получить у преподавателя) вариант задачи технического зрения и метод для его решения.

Создать команду для реализации решения выбранной задачи (проекта).

Команда - 1 или 2 человека по согласованию с преподавателем.

Проектное обучение базируется на следующих правилах и принципах работы в команде:

- все члены команды равны;
- команды не соревнуются;
- все члены команды должны получать удовольствие от общения друг с другом и от того, что они вместе выполняют проектное задание;
- каждый должен получать удовольствие от чувства уверенности в себе;
- все должны проявлять активность и вносить свой вклад в общее дело.

Не должно быть так называемых “спящих партнеров”;

- ответственность за конечный результат несут все члены команды, выполняющие проектное задание.

- умений ориентироваться в информационном пространстве.

Этап постановки задачи:

Концептуально (на человеческом языке) описать выбранную задачу технического зрения.

Провести анализ научно технической литературы (в интернете) по методам решения выбранной задачи технического зрения и провести классификацию методов.

Выбрать (получить у преподавателя) свой метод решения выбранной задачи технического зрения, показать достоинства и недостатки.

Провести анализ научно технической литературы (в интернете) по методу решения выбранной задачи технического зрения и кратко описать его основные этапы.

Также на данном этапе необходимо ответить на вопросы:

- как определить решение;
- каких данных не хватает и все ли они нужны;
- какие сделаны допущения и т.п.

Таким образом, кратко можно сказать, что на этапе постановки задачи необходимо:

- описание исходных данных и результата;
- формализация задачи;
- описание поведения программы в особых случаях (если таковые есть).

В ходе этой работы выявляются свойства, которыми должна обладать система в конечном виде (замысел), описываются функции системы, характеристики интерфейса.

2. Лабораторная работа 2.(4 часа)

Выявить и описать все этапы решения задачи технического зрения выбранным методом.

Распределение этапов решения задачи технического зрения выбранным методом членам команды.

Написать (найти в научно-технической литературе) математические модели для всех этапов решения задачи технического зрения выбранным методом.

3. Лабораторная работа 3.(4 часа)

Разработка общего алгоритма решения задачи и построение частных алгоритмов решения этапов задачи.

Разработка алгоритма – самый сложный и трудоемкий процесс, но и самый интересный в творческом отношении. Выбор метода разработки зависит от постановки задачи, ее модели.

При построении алгоритма для сложной задачи используют системный подход с использованием декомпозиции (нисходящее проектирование сверху-

вниз) и синтеза (программирование снизу-вверх). Как и при разработке структуры любой сложной системы, при формировании алгоритма используют дедуктивный и индуктивный методы.

При дедуктивном подходе рассматривается частный случай общеизвестных алгоритмических моделей. Здесь при заданных предположениях известный алгоритм приспособляется к условиям решаемой задачи. Например, многие вычислительные задачи линейной алгебры, в частности, нелинейные уравнения, системы алгебраических уравнений и т. п., могут быть решены с использованием известных методов и алгоритмов, для которых существует множество специальных библиотек подпрограмм, модулей. В настоящее время получили распространение специализированные пакеты, позволяющие решать многие задачи (Mathcad, Autocad и т. п.).

Индуктивный способ предполагает эвристический системный подход (декомпозиция - анализ - синтез). В этом случае общих и наиболее удачных методов не существует. Возможны некоторые подходы, позволяющие в каждом конкретном случае находить и строить алгоритмы. Методы разработки алгоритмов можно разбить на методы частных целей, подъема, отрабатывания назад, ветвей и границ и т. п.

Одним из системных методов разработки алгоритмов является структурное программирование.

Структурное программирование основано на использовании блок-схем, формируемых с помощью управляющих структурных элементов.

Выделяют три базовых структурных элемента (управляющие структуры): композицию, альтернативу, итерацию.

Композиция - это линейная конструкция алгоритма, составленная из последовательно следующих друг за другом функциональных вершин.

Альтернатива - это конструкция ветвления, имеющая предикатную вершину. Конструкция ветвления в алгоритмах может быть представлена в виде развилки.

Итерация - это циклическая конструкция алгоритма, которая, вообще говоря, является составной структурой, состоящей из композиции и альтернативы. Итерации могут быть представлены в двух формах: с предусловием и с постусловием.

Каждая из рассмотренных структур имеет один вход и один выход. Поэтому любая компьютерная программа может быть представлена блок-схемой, сформированной из представленных трех управляющих структур.

Процесс структурного программирования обычно начинается с разработки блок-схемы.

Идея структурного программирования сверху-вниз предполагает процесс пошагового разбиения алгоритма (блок-схемы) на все более мелкие части до уровня элементарных конструкций, для которых можно составить конкретные команды.

Для иллюстрации технологии структурного программирования сверху-вниз рассмотрим пример.

Пример. Технология разработки программы решения квадратного уравнения (рис. 2).

На рис. 2 проиллюстрирована пошаговая детализация процесса построения алгоритма. Заметим, что для начального шага разработки программы имеем в качестве входных данных коэффициенты a , b , c квадратного уравнения $ax^2 + bx + c = 0$, а на выходе - значения двух корней x_1 , x_2 .

Зачастую используют альтернативный процедуре сверху-вниз метод структурного программирования снизу-вверх. Фактически мы приходим к конечному результату системным методом. Сначала разбиваем задачу на отдельные блоки (модули) с их связями между собой (декомпозиция), затем, после их разработки, проводим сборку блоков в единую программу (синтез). Принцип снизу-вверх широко распространен среди программистов, которые предпочитают модульный подход, предполагающий максимальное

использование стандартных и специализированных библиотек процедур, функций, модулей и объектов.

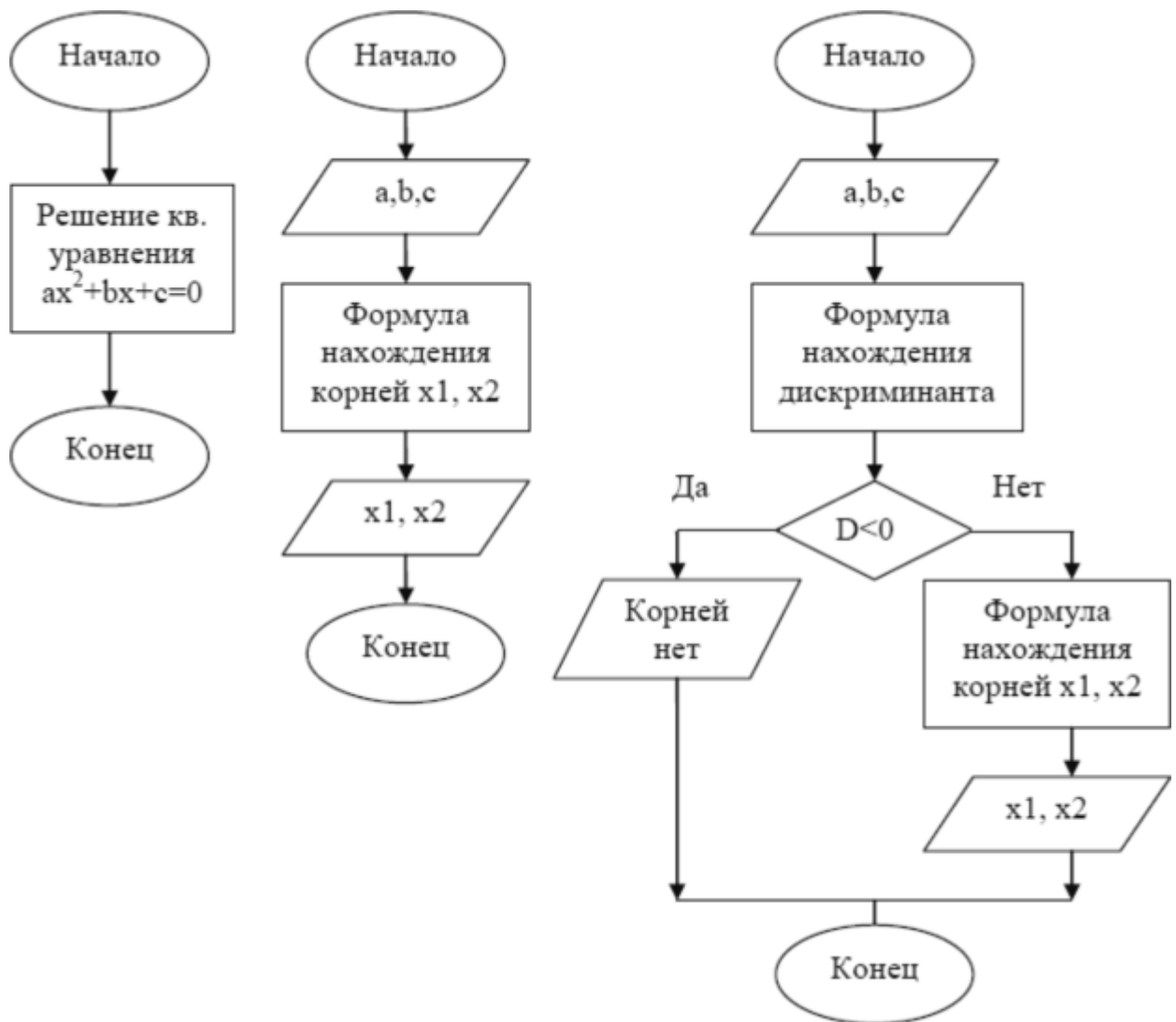


Рис. 2. Пример пошаговой детализации построения алгоритма

4. Лабораторная работа 4. (4 часа)

Разработка архитектуры системы (программы) или высокоуровневое проектирование.

Архитектура системы – это каркас программы, это высокоуровневое проектирование программы.

Данный этап также очень важный, так как, не имея хорошей архитектуры, вы можете решать правильную проблему, но прийти к

неправильному решению. Хорошая архитектура программы упрощает программирование, а плохая архитектура усложняет его.

Архитектура системы обычно включает:

Общее описание системы;

Основные компоненты;

Формат и способ хранения данных;

Специфические бизнес-правила;

Способ организации пользовательского интерфейса;

Подход к безопасности системы;

Оценки производительности;

Возможности масштабирования;

Кроме того, в архитектуру необходимо включить подтверждение того, что при разработке этой архитектуры рассматривались альтернативные варианты в каждом из вышеперечисленных направлений, с обоснованием окончательного выбора и подхода.

5. Лабораторные работы 5-6. (8 часов)

Выбор языка программирования.

Написание модулей программы (для разных этапов) по разработанным алгоритмам на выбранном языке программирования.

Объединение программных модулей в единую программу для решения задачи технического зрения.

Разработка интерфейса (интерфейс - способ общения) - создание экранной формы (окна программы).

6. Лабораторные работы 7-8 (8 часов).

Тестирование готового программного модуля для решения выбранной задачи технического зрения определённым методом.

Сравнительный анализ получаемых результатов для разных (не менее 5) примеров.

Защита проекта. Участие всех членов команды в рамках своих этапов.
Ответы на вопросы.

Создание документации – инструкции пользователя.

Если результаты тестирования, анализа и защиты не удовлетворительные, то необходимо провести отладку программы и отладку разработчиков.

Отладка программы - это процесс обнаружения и исправления ошибок. Программные ошибки можно разделить на два класса: синтаксические (синтаксис языка программирования) и алгоритмические (логические). Синтаксические ошибки выявляются в процессе компилирования программы – это наиболее простые с точки зрения исправления ошибки. Алгоритмические ошибки программы выявить гораздо труднее: программа работает, а результат выдает неправильный. Для обнаружения ошибок этого класса требуется специальные этапы тестирования программы.

Повторная защита проекта. Участие всех членов команды в рамках своих этапов. Ответы на вопросы.

Создание исправленной документации – инструкции пользователя.

Примерные варианты тем для лабораторных работ:

1. Разработка программы для выделения контуров на изображении методом на основе оператора Робертса.
2. Разработка программы для выделения контуров на изображении методом на основе оператора Превитта.
3. Разработка программы для выделения контуров на изображении методом на основе оператора Собеля.
4. Разработка программы для выделения контуров на изображении методом на основе оператора Кирша.
5. Разработка программы для выделения контуров на изображении методом на основе оператора Робинсона.
6. Разработка программы для выделения контуров на изображении детектором границ Кэнни.
7. Разработка программы для выделения контуров на изображении методом Марра-Хилдрет.
8. Разработка программы для выделения контуров на изображении ISEF детектором.
9. Разработка программы для выделения контуров на изображении на основе вейвлет-преобразования.
10. Разработка программы для выделения контуров на изображении методом на основе генетических алгоритмов.
11. Разработка программы для обнаружения лиц (автомобильных номеров, дорожных знаков или др.) на изображении методом Виолы-Джонса.
12. Разработка программы для идентификации человека по радужной оболочке глаза (выбрать метод).
13. Разработка программы для идентификации человека по лицу (выбрать метод).
14. Разработка программы для идентификации человека по отпечатку пальца (выбрать метод).

15. Разработка программы для идентификации человека по подписи (выбрать метод).
16. Разработка программы для распознавания текста (выбрать метод).
17. Разработка программы для анализа движения транспортных средств (людей) на изображениях (выбрать метод).
18. Разработка программы для классификации изображений на основе наивного Байесовского классификатора.
19. Разработка программы для классификации изображений на основе структурного распознавания.
20. Разработка программы для графовой кластеризации данных.
21. Разработка программы для иерархической кластеризации данных.
22. Разработка программы для геометрической аугментации изображений.

Список литературы:

1. Селянкин, В. В. Компьютерное зрение. Анализ и обработка изображений : учебное пособие / В. В. Селянкин. — Санкт-Петербург : Лань, 2019. — 152 с. — ISBN 978-5-8114-3368-1. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/113938> (дата обращения: 07.02.2022). — Режим доступа: для авториз. пользователей.
2. Флах, П. Машинное обучение. Наука и искусство построения алгоритмов, которые извлекают знания из данных [Электронный ресурс]: учебник / П. Флах. Москва : ДМК Пресс, 2015. 400 с. URL: <https://e.lanbook.com/book/69955> (дата обращения: 15.01.2019). Режим доступа: для авториз. пользователей.
3. Научно-техническая библиотека КНИТУ-КАИ [Электронный ресурс]. URL: <http://library.kai.ru/> (дата обращения: 15.01.2019). Режим доступа: свободный.
4. Лань : электронно-библиотечная система [Электронный ресурс] . URL: <https://e.lanbook.com> (дата обращения: 15.01.2019). Режим доступа: для авториз. пользователей.
5. Воронцов, К.В. Введение в машинное обучение [Электронный ресурс] / К.В. Воронцов, Е.А. Соколов // Coursera. URL: <https://www.coursera.org/learn/vvedenie-mashinnoe-obuchenie/home/info> (дата обращения: 15.01.2019). Режим доступа: для авториз. пользователей.
6. [https://ru.wikipedia.org/wiki/компьютерное зрение](https://ru.wikipedia.org/wiki/компьютерное_зрение)
7. Шапиро Л. Компьютерное зрение / Дж. Стокман. - Москва: Лаборатория знаний, 2020 .— 763 с.
8. Костюхина Г.В. Модель, метод и комплекс программ выделения контуров на изображениях с использованием энергетических признаков: диссер. на соиск. степ. к.т.н., 2020 URL: https://old.kai.ru/science/disser/files/file_311/synopsis.pdf

9. Рашка, С. Python и машинное обучение: машинное и глубокое обучение с использованием Python, scikit-learn и TensorFlow: Изд. 2-е; пер. с англ. / С. Рашка, В. Мирджалили. – СПб.: ООО «Диалектика», 2019.

10. Лучано, Р. Python. К вершинам мастерства [Электронный ресурс] / Р. Лучано. Москва : ДМК Пресс, 2016. 768 с. URL: <https://e.lanbook.com/book/93273> (дата обращения: 15.01.2019). Режим доступа: для авториз. пользователей.

11. Сузи, Р.А. Язык программирования Python [Электронный ресурс] : учебное пособие / Р.А. Сузи. Москва : ИНТУИТ, 2016. 350 с. URL: <https://e.lanbook.com/book/100546> (дата обращения: 15.01.2019). Режим доступа: для авториз. пользователей.

12. Плас, Д.В. Python для сложных задач: наука о данных и машинное обучение : учебник / Д.В. Плас. Санкт-Петербург : Питер, 2018. 576 с.